1 / 8

Attention combines weighted values.

Follow query zero and one component of its four-component output.

> Normalised weights combine every value row.

Query 0 · output component 0

Key	Score	Weight	V[k,0]	Product
0	-1.75	0.010187	-4.0	-0.040748
1	-1.0	0.021566	-3.0	-0.064697
2	1.5	0.262724	-2.0	-0.525447
3	0.5	0.096651	-1.0	-0.096651
4	-2.25	0.006179	0.0	0.0
5	2.0	0.433158	1.0	0.433158
6	1.0	0.15935	2.0	0.3187
7	-1.75	0.010187	3.0	0.030561

Weighted component: 0.054876

Scores, weights and products are rounded for display.

Copyright © 2026 Christopher Snow

2 / 8

Full intermediates need reads and writes.

The straightforward calculation stores scores and weights as full matrices.

> Selected intermediate accounting, not measured DRAM traffic.

Two full N × N intermediate matrices

Scores

8 × 8 entries

Weights

8 × 8 entries

128 writes + 128 reads · 4 bytes each

1,024 selected intermediate bytes

Copyright © 2026 Christopher Snow

3 / 8

Keep totals while visiting key tiles.

Start with just keys zero and one.

> Only this tile’s scores are needed for the next update.

First tile: keys 0–1

Largest score m = -1.0

Key	Score	exp(score – m)
0	-1.75	0.472367
1	-1.0	1.0

Normalisation l = 1.472367

Weighted a_o = -4.889466

a_o / l = -3.320821

An estimate from two keys · four weighted totals exist in all.

Copyright © 2026 Christopher Snow

4 / 8

A later larger score changes the scale.

Both retained totals must move to the same new scale.

> Rescale the retained numerator and denominator together.

Query 0 · keys 2–3

Largest score: -1.0 → 1.5

Factor = exp(old − new) = 0.082085

Quantity	Previous total	After rescaling
Normalisation l	**1.472367**	**0.120859**
Weighted a_0	**-4.889466**	**-0.401352**

Tile adds: l 1.367879 · a_0 -2.367879

New totals: l 1.488739 · a_0 -2.769231

$a_0 / l = -1.860119$

The estimate includes the keys visited so far.

Copyright © 2026 Christopher Snow

5 / 8

Include every key before taking the final result.

The running estimate can change until all eight keys contribute.

> Final comparison uses the same inputs and all queries.

Query 0 visits all four key tiles

Keys 0–1	m -1.0	factor 0.0
Keys 2–3	m 1.5	factor 0.082085
Keys 4–5	m 2.0	factor 0.606531
Keys 6–7	m 2.0	factor 1.0

0.126688 / 2.308627 = 0.054876

Reference component: 0.054876

All 8 output vectors checked · maximum difference 4.44089e-16

Copyright © 2026 Christopher Snow

6 / 8

Count the scratch for one query.

Two scores, four weighted totals and two normalisation scalars.

> Single-query scratch excludes Q/K/V, output and tile rereads.

Algorithmic state for one query

score 0	score 1	weighted a0	weighted a1
weighted a2	weighted a3	maximum m	normalisation l

2 scores + 4 weighted totals + 2 scalars

8 floats × 4 bytes = 32 bytes

0 full-matrix intermediate bytes · inputs/outputs excluded

Copyright © 2026 Christopher Snow

7 / 8

Pause: use four keys per tile.

Same queries, keys and values. A different grouping.

> Predict before comparing the updated totals.

Four keys per tile · same Q, K and V

First tile scores

-1.75, -1.0, 1.5, 0.5

Second tile scores

-2.25, 2.0, 1.0, -1.75

Do both retained totals need the same new scale?

Will the final weighted result change?

Scratch rule: tile width + four components + two scalars

Copyright © 2026 Christopher Snow

8 / 8

Regrouping preserves the final calculation.

Both totals rescale; the scratch budget grows to ten floats.

> Same calculation; grouping changes intermediate steps and scratch.

Query 0 · keys 4–7

Largest score: 1.5 → 2.0

Factor = exp(old − new) = 0.606531

Quantity	Previous total	After rescaling
Normalisation l	**1.488739**	**0.902966**
Weighted a_0	**-2.769231**	**-1.679624**

Tile adds: l 1.405661 · a_0 1.806312

New totals: l 2.308627 · a_0 0.126688

$a_0 / l = 0.054876$

10 scratch floats × 4 bytes = 40 bytes · same final calculation

Copyright © 2026 Christopher Snow